100 C Aptitude Questions & Answers — Zoho Round 1 Prep

Zoho's Round 1 (written test) typically has **output-prediction / debugging style C questions** covering pointers, arrays, strings, operators, structures, storage classes, and memory. These 100 questions are grouped by topic, in that style.

Section 1: Operators & Expressions (Q1–15)

Q1. What is the output?

```
int a = 5, b = 2;

printf("%d", a/b);
```

Ans: `2` **Explanation:** Both operands are `int`, so `/` performs integer division, truncating the decimal part (2.5 → 2).

Q2.

```
int a = 5;

printf("%d %d %d", a++, ++a, a);
```

Ans: Undefined behavior — most compilers (order of evaluation of function args is unspecified) may print `6 7 7` or similar. **Explanation:** Modifying `a` more than once between sequence points without a defined order makes this undefined behavior. Never rely on a specific output here in real code — but in written tests, GCC commonly evaluates right-to-left, giving `6 7 7` for `printf`.

Q3.

```
int x = 10;
```

x = x++ + ++x;

printf("%d", x);

Ans: Undefined behavior (compiler-dependent, often `22` in GCC). **Explanation:** `x` is modified twice without an intervening sequence point — classic UB trap question.

Q4.

printf("%d", 5 == 5 == 1);

Ans: `1` **Explanation:** `==` is left-associative: `(5==5)` → `1`, then `1==1` → `1`.

Q5.

int a = 5;

printf("%d", a > 3 ? 10 : 20);

Ans: `10` **Explanation:** Ternary operator: condition `a>3` (true), so the first expression is returned.

Q6.

printf("%d", 10 % 3);

printf("%d", -10 % 3);

Ans: `1` then `-1` **Explanation:** In C, the sign of the result of `%` follows the sign of the dividend (numerator), not the divisor.

Q7.

int a = 1, b = 2, c = 3;

printf("%d", a && b || c);

Ans: 1 **Explanation:** && has higher precedence than ||. (1 && 2) → 1; 1 || 3 → 1.

Q8.

int x = 5;

printf("%d", x = x > 4);

Ans: 1 **Explanation:** x > 4 evaluates to 1 (true); this is assigned to x, and printf prints the new value of x.

Q9.

printf("%d", sizeof(5.0));

Ans: 8 **Explanation:** Floating point literals like 5.0 are of type double by default (8 bytes on most systems), unless suffixed with f.

Q10.

printf("%d", sizeof('A'));

Ans: 4 (in C; would be 1 in C++) **Explanation:** Character constants are of type int in C (unlike C++ where they are char), so sizeof('A') equals sizeof(int).

Q11.

int a = 3;

int b = a << 2;

printf("%d", b);

Ans: 12 **Explanation:** Left shift by 2 multiplies by 2^2 = 4. 3 << 2 = 3 * 4 = 12.

Q12.

int a = 5, b = 10;

a = a + b - (b = a);

printf("%d %d", a, b);

Ans: Undefined/compiler-dependent, commonly `5 5` in GCC. **Explanation:** `b` is both read and modified in the same expression without a sequence point — classic UB.

Q13.

printf("%d", 1 & 2);

printf("%d", 1 | 2);

Ans: `0` then `3` **Explanation:** Bitwise AND of `01` & `10` = `00` = 0. Bitwise OR of `01` | `10` = `11` = 3.

Q14.

int x = 10;

printf("%d", !x);

printf("%d", !!x);

Ans: `0` then `1` **Explanation:** Logical NOT converts any non-zero value to `0`, and `0` to `1`. Double negation restores a boolean-like value.

Q15.

float f = 5/2;

printf("%f", f);

Ans: `2.000000` **Explanation:** `5/2` is computed first as integer division (`2`), and only then converted to `float` when stored — the fractional part is already lost.

Section 2: Pointers (Q16–35)

Q16.

int a = 10;

int *p = &a;

printf("%d", *p);

Ans: `10` **Explanation:** `p` stores the address of `a`; dereferencing `*p` gives the value stored at that address.

Q17.

int a = 10;

int *p = &a;

*p = 20;

printf("%d", a);

Ans: `20` **Explanation:** Modifying `*p` modifies the value at the address `p` points to, which is `a` itself.

Q18.

int arr[5] = {1,2,3,4,5};

printf("%d", *(arr + 2));

Ans: `3` **Explanation:** Array name decays to a pointer to its first element; `arr + 2` points to `arr[2]`, and dereferencing gives `3`.

Q19.

int arr[5] = {1,2,3,4,5};

printf("%d", *(arr + 2) == arr[2]);

Ans: `1` **Explanation:** `*(arr + i)` and `arr[i]` are equivalent in C by definition of pointer arithmetic.

Q20.

int a = 5;

int *p = &a;

int **q = &p;

printf("%d", **q);

Ans: `5` **Explanation:** `q` is a pointer to a pointer; `*q` gives `p` (the address of `a`), and `**q` dereferences again to get `a`'s value.

Q21.

char str[] = "Hello";

char *p = str;

printf("%c", *(p+1));

Ans: `e` **Explanation:** `p` points to `str[0]`; `*(p+1)` accesses `str[1]`, which is `'e'`.

Q22.

```
int a = 10, b = 20;

int *p = &a, *q = &b;

p = q;

printf("%d", *p);
```

Ans: `20` **Explanation:** After `p = q`, `p` now holds the address of `b`, so dereferencing gives `20`.

Q23.

```
void func(int *p) {
    *p = 100;
}

int main() {
    int a = 5;
    func(&a);
    printf("%d", a);
}
```

Ans: `100` **Explanation:** Passing the address allows the function to modify the original variable — this is "call by reference" simulated using pointers.

Q24.

```
int a[] = {1,2,3};

int *p = a;
```

p++;

printf("%d", *p);

Ans: `2` **Explanation:** `p` initially points to `a[0]`. `p++` advances the pointer by `sizeof(int)` bytes, now pointing to `a[1]`.

Q25.

int a = 10;

void *p = &a;

printf("%d", *(int*)p);

Ans: `10` **Explanation:** A `void*` cannot be dereferenced directly; it must first be cast to a proper type (`int*` here) before dereferencing.

Q26.

const int *p;

int a = 5, b = 10;

p = &a;

p = &b;

printf("%d", *p);

Ans: `10` **Explanation:** `const int *p` means the **value pointed to** is constant (can't do `*p = x`), but `p` itself can be reassigned to point elsewhere.

Q27.

int a = 5;

```
int * const p = &a;

a = 10;

printf("%d", *p);
```

Ans: `10` **Explanation:** `int * const p` makes the pointer itself constant (can't repoint `p`), but the value it points to can still be modified directly through `a`.

Q28.

```
int arr[3][3] = {{1,2,3},{4,5,6},{7,8,9}};

printf("%d", *(*(arr+1)+2));
```

Ans: `6` **Explanation:** `arr+1` points to row 1 (`{4,5,6}`); `*(arr+1)` decays to a pointer to that row; adding 2 and dereferencing gives element `arr[1][2] = 6`.

Q29.

```
int a = 10;

int *p = NULL;

printf("%d", p == NULL);
```

Ans: `1` **Explanation:** `p` was explicitly initialized to `NULL`, so the comparison evaluates true (1).

Q30.

```
char *s = "Zoho";

printf("%s", s);

s = "Hello";

printf("%s", s);
```

Ans: `ZohoHello` **Explanation:** `s` is a pointer that can be reassigned to point to a different string literal; both literals live in read-only memory.

Q31.

int a = 5;

int *p = &a;

(*p)++;

printf("%d", a);

Ans: `6` **Explanation:** `(*p)++` increments the value pointed to by `p`, which is `a`. Parentheses are required — `*p++` would instead increment the pointer.

Q32.

int arr[5] = {10,20,30,40,50};

int *p = &arr[4];

printf("%d", *p--);

printf("%d", *p);

Ans: `50` then `40` **Explanation:** Post-decrement means the current value is used first, then `p` is decremented. First `*p` gives `50` (postfix), then `p` moves back one element, giving `40`.

Q33.

int x = 5;

int *p = &x;

printf("%p == %p", (void*)p, (void*)&x);

Ans: Same address printed twice (identical). **Explanation:** `p` was initialized as `&x`, so it holds exactly the same address as `&x`.

Q34.

int a[5] = {1,2,3,4,5};

printf("%d", sizeof(a)/sizeof(a[0]));

Ans: `5` **Explanation:** `sizeof(a)` gives total bytes of the array (5×4=20 on most systems); dividing by `sizeof(a[0])` (4) gives the number of elements.

Q35.

int a = 5;

int *p = &a;

int *q = p;

p = NULL;

printf("%d", *q);

Ans: `5` **Explanation:** `q` was assigned a **copy** of `p`'s value (the address of `a`) at the time of assignment. Later changing `p` to `NULL` doesn't affect `q`.

Section 3: Arrays & Strings (Q36–55)

Q36.

char str[] = "Zoho";

printf("%d", strlen(str));

Ans: `4` **Explanation:** `strlen` counts characters up to (not including) the null terminator `'\0'`.

Q37.

char str[] = "Zoho";

printf("%d", sizeof(str));

Ans: `5` **Explanation:** `sizeof` on a char array includes the implicit null terminator, so it's `strlen + 1`.

Q38.

char a[10] = "Hi";

char b[10] = "Hi";

printf("%d", a == b);

Ans: `0` **Explanation:** `a` and `b` are separate array objects at different memory addresses; comparing array names compares their (different) addresses, not contents.

Q39.

char a[] = "Zoho";

char b[] = "Zoho";

printf("%d", strcmp(a,b));

Ans: `0` **Explanation:** `strcmp` returns `0` when the strings' contents are identical, regardless of their memory addresses.

Q40.

char str[100] = "Hello";

strcat(str, " World");

printf("%s", str);

Ans: `Hello World` **Explanation:** `strcat` appends the second string to the end of the first, requiring the destination to have enough space.

Q41.

char str[] = "abcdef";

printf("%c", str[10]);

Ans: Undefined behavior (garbage value / crash). **Explanation:** Accessing `str[10]` is out-of-bounds for a 7-byte array (`abcdef\0`), leading to undefined behavior.

Q42.

int arr[] = {1,2,3,4,5};

int n = sizeof(arr)/sizeof(arr[0]);

for(int i=0;i<n/2;i++){

    int t = arr[i];

    arr[i] = arr[n-1-i];

    arr[n-1-i] = t;

}

for(int i=0;i<n;i++) printf("%d ", arr[i]);

Ans: `5 4 3 2 1` **Explanation:** This is a standard in-place array reversal using a swap loop that runs to the middle of the array.

Q43.

char str[] = "Zoho Corp";

printf("%c", *str);

printf("%c", str[0]);

Ans: `ZZ` **Explanation:** `*str` and `str[0]` are equivalent — dereferencing the array's decayed pointer gives the first character.

Q44.

char a[5] = "abcd";

printf("%s", a);

Ans: `abcd` **Explanation:** `"abcd"` has 4 characters + null terminator = exactly 5 bytes, fitting perfectly into `a[5]`; the string is correctly terminated.

Q45.

char *ptr = "Hello";

ptr[0] = 'M';

printf("%s", ptr);

Ans: Undefined behavior (often a crash / segmentation fault). **Explanation:** String literals are stored in read-only memory. Modifying them through a `char*` pointer causes undefined behavior on most systems (unlike a `char array[]`, which is modifiable).

Q46.

int a[3] = {1,2,3};

int b[3];

b = a;

Ans: Compile-time error. **Explanation:** Arrays are not assignable in C (array names are not modifiable lvalues); you must copy elements individually or use `memcpy`.

Q47.

char str[20];

gets(str); // input: Zoho

printf("%s", str);

Ans: `Zoho` **Explanation:** `gets` reads a line from input into `str` (deprecated/unsafe due to no bounds checking — `fgets` is preferred in modern code, a point interviewers often ask about).

Q48.

int arr[5];

printf("%d", arr[5]);

Ans: Undefined behavior. **Explanation:** Valid indices for a 5-element array are 0–4; `arr[5]` reads out-of-bounds memory.

Q49.

char str[] = "racecar";

int i=0, j=strlen(str)-1, flag=1;

while(i<j){

  if(str[i]!=str[j]) { flag=0; break; }

```
    i++; j--;
}
printf("%s", flag ? "Palindrome" : "Not Palindrome");
```

Ans: `Palindrome` **Explanation:** Classic two-pointer palindrome check — compares characters from both ends moving inward; "racecar" reads the same forwards and backwards.

Q50.

```
int a[2][3] = {1,2,3,4,5,6};
printf("%d", a[1][2]);
```

Ans: `6` **Explanation:** In row-major order, `a[1][2]` is the 3rd element of the 2nd row: values fill as `{1,2,3}` (row 0) then `{4,5,6}` (row 1), so `a[1][2] = 6`.

Q51.

```
char s1[] = "abc", s2[] = "abd";
printf("%d", strcmp(s1,s2));
```

Ans: Negative value (e.g., `-1`) **Explanation:** `strcmp` compares lexicographically; `'c' < 'd'`, so `s1` is "less than" `s2`, giving a negative result.

Q52.

```
char str[] = "Hello";
printf("%d", strlen(str) == sizeof(str));
```

Ans: `0` **Explanation:** `strlen` = 5 (excludes null terminator); `sizeof` = 6 (includes it) — they're not equal.

Q53.

```
int arr[] = {5, 2, 8, 1, 9};

int max = arr[0];

for (int i = 1; i < 5; i++)

    if (arr[i] > max) max = arr[i];

printf("%d", max);
```

Ans: `9` **Explanation:** Standard linear scan to find the maximum element by comparing each element against the running maximum.

Q54.

```
char str[10] = "abc";

strcpy(str, "defgh");

printf("%s", str);
```

Ans: `defgh` **Explanation:** `strcpy` overwrites the destination buffer's contents entirely (and its terminator), as long as the buffer is large enough.

Q55.

```
int arr[3] = {10, 20, 30};

int *p = arr;

printf("%d %d %d", p[0], p[1], p[2]);
```

Ans: `10 20 30` **Explanation:** Pointer indexing `p[i]` is equivalent to `*(p+i)`, working the same as array indexing since `p` points to the array's start.

Section 4: Functions & Recursion (Q56–70)

Q56.

```
int fact(int n) {
    if (n == 0) return 1;
    return n * fact(n-1);
}
int main() { printf("%d", fact(5)); }
```

Ans: `120` **Explanation:** Classic recursive factorial: `5*4*3*2*1*1 = 120`, with base case `fact(0)=1`.

Q57.

```
int fib(int n) {
    if (n <= 1) return n;
    return fib(n-1) + fib(n-2);
}
int main() { printf("%d", fib(6)); }
```

Ans: `8` **Explanation:** Fibonacci sequence: 0,1,1,2,3,5,8 — `fib(6)` is the 7th term (0-indexed), which is `8`.

Q58.

```
void swap(int a, int b) {
    int t = a; a = b; b = t;
```

```
}

int main() {

    int x=5, y=10;

    swap(x,y);

    printf("%d %d", x, y);

}
```

Ans: `5 10` **Explanation:** C is "pass by value" — `swap` operates on **copies** of `x` and `y`, so the originals in `main` remain unchanged.

Q59.

```
void swap(int *a, int *b) {

    int t = *a; *a = *b; *b = t;

}

int main() {

    int x=5, y=10;

    swap(&x,&y);

    printf("%d %d", x, y);

}
```

Ans: `10 5` **Explanation:** Passing addresses lets the function dereference and modify the original variables directly — simulated "pass by reference."

Q60.

```
int square(int x) {
```

```
    return x * x;
}
int main() {
    int (*fp)(int) = square;
    printf("%d", fp(5));
}
```

Ans: `25` **Explanation:** `fp` is a function pointer assigned to `square`; calling `fp(5)` calls `square(5)`, returning `25`.

Q61.

```
int add(int a, int b) {
    return a + b;
}
int main() {
    printf("%d", add(2,3,4));
}
```

Ans: Compile-time error (or warning + only first two args used, depending on compiler).
Explanation: In standard C (with a prototype declared), calling a function with more arguments than it's declared to take is a compile error since C99 requires matching prototypes.

Q62.

```
int count = 0;
void func() {
```

```
    static int c = 0;
    c++;
    printf("%d ", c);
}
int main() {
    func(); func(); func();
}
```

Ans: `1 2 3` **Explanation:** `static` local variables retain their value across function calls, unlike normal locals which reinitialize each call.

Q63.

```
int func() {
    int x = 10;
    return x;
}
int main() {
    int *p = NULL;
    { int y = func(); p = &y; }
    printf("%d", *p);
}
```

Ans: Undefined behavior. **Explanation:** `y` is a local variable inside the inner block; once the block ends, `y` goes out of scope and `p` becomes a dangling pointer.

Q64.

```
void func(int n) {
    if (n == 0) return;
    printf("%d ", n);
    func(n-1);
}
int main() { func(5); }
```

Ans: `5 4 3 2 1` **Explanation:** Each call prints `n` before recursing, so numbers print in descending order until the base case `n==0` stops recursion.

Q65.

```
void func(int n) {
    if (n == 0) return;
    func(n-1);
    printf("%d ", n);
}
int main() { func(5); }
```

Ans: `1 2 3 4 5` **Explanation:** Since the recursive call happens **before** the print statement, the deepest call finishes first, printing in ascending order as the recursion unwinds.

Q66.

```
int func(int a, int b) {
```

```
    return a > b ? a : b;

}

int main() { printf("%d", func(3,7)); }
```

Ans: `7` **Explanation:** Simple max function using ternary — returns whichever argument is larger.

Q67.

```
int global = 100;

void func() {

    int global = 10;

    printf("%d", global);

}

int main() { func(); }
```

Ans: `10` **Explanation:** The local `global` inside `func` **shadows** the global variable of the same name within its scope.

Q68.

```
int func(int x) {

    x = x + 5;

    return x;

}

int main() {
```

```
    int a = 10;
    printf("%d", func(a));
    printf("%d", a);
}
```

Ans: `1510` **Explanation:** `func` receives a copy of `a`; modifying the parameter inside the function doesn't affect the caller's `a`.

Q69.

```
int power(int base, int exp) {
    if (exp == 0) return 1;
    return base * power(base, exp-1);
}
int main() { printf("%d", power(2,5)); }
```

Ans: `32` **Explanation:** Recursive exponentiation: `2*2*2*2*2*1 = 32`.

Q70.

```
int main() {
    int a = 10;
    void func(int);
    func(a);
    printf("%d", a);
}
```

```
void func(int x) { x *= 2; }
```

Ans: `10` **Explanation:** Even with a forward-declared prototype, C is still pass-by-value; modifying `x` inside `func` doesn't affect `a` in `main`.

Section 5: Structures, Unions & typedef (Q71–80)

Q71.

```
struct Point { int x, y; };

int main() {
    struct Point p1 = {10, 20};
    struct Point p2 = p1;
    p2.x = 100;
    printf("%d %d", p1.x, p2.x);
}
```

Ans: `10 100` **Explanation:** Struct assignment (`p2 = p1`) copies all members by value; `p1` and `p2` become independent, so modifying `p2` doesn't affect `p1`.

Q72.

```
struct Point { int x, y; };

void modify(struct Point p) {
    p.x = 999;
}

int main() {
    struct Point p1 = {10, 20};
```

```
    modify(p1);

    printf("%d", p1.x);

}
```

Ans: `10` **Explanation:** The struct is passed by value, so `modify` works on a local copy; the original `p1` is unaffected.

Q73.

```
struct Point { int x, y; };

void modify(struct Point *p) {

    p->x = 999;

}

int main() {

    struct Point p1 = {10, 20};

    modify(&p1);

    printf("%d", p1.x);

}
```

Ans: `999` **Explanation:** Passing the address lets `modify` dereference and change the original struct via the `->` operator.

Q74.

```
union Data { int i; float f; char c; };

int main() {
```

```
    union Data d;

    d.i = 65;

    printf("%c", d.c);

}
```

Ans: `A` **Explanation:** In a union, all members share the same memory location. Setting `d.i = 65` and reading the first byte as `d.c` interprets that byte as ASCII `'A'` (65) — valid on little-endian systems.

Q75.

```
struct A { int x; };

struct B { int x; };

int main() {

    printf("%d", sizeof(struct A) == sizeof(struct B));

}
```

Ans: `1` **Explanation:** Both structures have a single `int` member, so they occupy the same amount of memory (typically 4 bytes), regardless of their different names/types.

Q76.

```
struct Point { int x, y; } p1 = {1, 2};

typedef struct Point Pt;

int main() {

    Pt p2 = {3, 4};

    printf("%d %d", p1.x, p2.x);
```

}

Ans: `1 3` **Explanation:** `typedef` just creates an alias `Pt` for `struct Point`; both refer to the same type, but `p1` and `p2` are separate variable instances.

Q77.

```
struct Emp { char name[20]; int age; };

int main() {

    struct Emp e = {"Ravi", 25};

    printf("%s is %d", e.name, e.age);

}
```

Ans: `Ravi is 25` **Explanation:** Struct initialization fills members in declaration order; `e.name` and `e.age` are accessed via the `.` operator.

Q78.

```
struct Node {

    int data;

    struct Node *next;

};

int main() {

    struct Node n1 = {10, NULL};

    printf("%d", n1.data);

}
```

Ans: `10` **Explanation:** A self-referential structure (common in linked lists) — `next` is a pointer to the same struct type; here it's just initialized to `NULL`.

Q79.

struct A { int x:4; int y:4; };

int main() { printf("%d", sizeof(struct A)); }

Ans: `4` (typically, though it can be compiler/platform dependent — often 4 due to `int` allocation unit) **Explanation:** Bit-fields `x:4` and `y:4` request 4 bits each; they're typically packed into a single storage unit (commonly padded to `sizeof(int)`), so the total is usually `4` bytes, not `1`.

Q80.

enum Color { RED, GREEN, BLUE };

int main() {

    enum Color c = GREEN;

    printf("%d", c);

}

Ans: `1` **Explanation:** Enum constants default to sequential integers starting from `0`; `RED=0, GREEN=1, BLUE=2`.

Section 6: Storage Classes, Preprocessor & Memory (Q81–100)

Q81.

#define SQUARE(x) x*x

int main() { printf("%d", SQUARE(3+2)); }

Ans: `11` **Explanation:** Macros do plain text substitution: `3+2*3+2` = `3 + 6 + 2` = `11` (not the intended `25`), because the macro argument isn't parenthesized — a classic macro pitfall.

Q82.

```
#define SQUARE(x) ((x)*(x))

int main() { printf("%d", SQUARE(3+2)); }
```

Ans: `25` **Explanation:** Wrapping both the macro and its parameter in parentheses fixes the precedence issue: `((3+2)*(3+2)) = 25`.

Q83.

```
int main() {

    static int x;

    printf("%d", x);

}
```

Ans: `0` **Explanation:** Static (and global) variables are automatically zero-initialized if no explicit initializer is given.

Q84.

```
int main() {

    int x;

    printf("%d", x);

}
```

Ans: Undefined/garbage value. **Explanation:** Local (automatic) variables are **not** automatically initialized; their initial value is indeterminate.

Q85.

```
#include <stdio.h>

#define PI 3.14

int main() { printf("%.2f", PI * 2); }
```

Ans: `6.28` **Explanation:** The preprocessor replaces `PI` with `3.14` before compilation; the expression becomes `3.14 * 2 = 6.28`.

Q86.

```
void func() {

   auto int x = 10;

   printf("%d", x);

}
```

Ans: `10` **Explanation:** `auto` is the default storage class for local variables (rarely written explicitly) — it behaves the same as a normal local variable.

Q87.

```
extern int x;

int main() { printf("%d", x); }

int x = 50;
```

Ans: `50` **Explanation:** `extern` tells the compiler `x` is defined elsewhere (later in this case); the linker resolves it to the actual definition, giving `50`.

Q88.

```
int *p = (int*) malloc(5 * sizeof(int));

p[0] = 10;

printf("%d", p[0]);

free(p);
```

Ans: `10` **Explanation:** `malloc` dynamically allocates memory for 5 integers on the heap; `p` can be indexed like an array, and `free` releases it after use.

Q89.

```
int *p = (int*) malloc(5 * sizeof(int));

free(p);

printf("%d", p[0]);
```

Ans: Undefined behavior (dangling pointer / garbage value / possible crash). **Explanation:** After `free(p)`, the memory is deallocated; accessing `p[0]` afterward is a **use-after-free** bug.

Q90.

```
int *p = (int*) calloc(5, sizeof(int));

printf("%d", p[2]);
```

Ans: `0` **Explanation:** Unlike `malloc`, `calloc` initializes all allocated memory to zero.

Q91.

```
register int x = 5;
```

int *p = &x;

Ans: Compile-time error (in strict/older compilers) or a warning in modern ones. **Explanation:** `register` suggests the variable be stored in a CPU register (no guaranteed memory address), so taking its address with `&` is technically illegal / discouraged — modern C mostly ignores this restriction but it's a classic conceptual question.

Q92.

```
#if 1
   #define VAL 100
#else
   #define VAL 200
#endif
int main() { printf("%d", VAL); }
```

Ans: `100` **Explanation:** Conditional compilation: since `#if 1` is always true, only the `VAL 100` branch is compiled into the code.

Q93.

```
int main() {
   int a = 10;
   {
      int a = 20;
      printf("%d ", a);
   }
   printf("%d", a);
```

}

Ans: `20 10` **Explanation:** The inner block declares a new `a` that shadows the outer one within its scope; once the block ends, the outer `a` (10) is visible again.

Q94.

```
int *ptr = (int*) malloc(sizeof(int) * 3);

if (ptr == NULL) printf("Allocation failed");

else printf("Allocation successful");

free(ptr);
```

Ans: `Allocation successful` (in typical conditions) **Explanation:** `malloc` returns `NULL` only if allocation fails (e.g., out of memory); it's good practice to always check the returned pointer before use.

Q95.

```
int x = 5;

int y = x++ + x++;

printf("%d %d", x, y);
```

Ans: Undefined behavior (commonly `7 10` with GCC, but not guaranteed). **Explanation:** `x` is modified twice without a sequence point between the accesses in the same expression — this is a classic UB trap seen frequently in these tests.

Q96.

```
int main() {

    int a[5] = {1,2,3,4,5};
```

```
    int *p = a + 5;

    printf("%d", *(p-1));

}
```

Ans: `5` **Explanation:** `a+5` points one past the last element (valid to compute but not to dereference directly); `*(p-1)` steps back to the last valid element, `a[4] = 5`.

Q97.

```
#define MAX(a,b) ((a)>(b)?(a):(b))

int main() {

    int i = 5;

    printf("%d", MAX(i++, 3));

    printf("%d", i);

}
```

Ans: Undefined behavior-ish result, but classically: `6` then `7` (since `i++` may be evaluated twice by macro expansion). **Explanation:** Macro arguments are substituted textually, so `i++` can appear twice in `((i++)>(3)?(i++):(3))`, evaluating `i++` more than once — a hidden side-effect bug distinct from a real function call.

Q98.

```
int main() {

    printf("%d", 'A' + 1);

}
```

Ans: `66` **Explanation:** Character constants are promoted to `int` in arithmetic; `'A'` is ASCII `65`, so `65 + 1 = 66`.

Q99.

```
int main() {
    char c = 'A' + 1;
    printf("%c", c);
}
```

Ans: `B` **Explanation:** The arithmetic `'A' + 1` gives `66`, which when stored/printed as a `char` corresponds to the ASCII character `'B'`.

Q100.

```
int main() {
    int i;
    for (i = 0; i < 3; i++) {
        int x = 0;
        x++;
        printf("%d ", x);
    }
}
```

Ans: `1 1 1` **Explanation:** `x` is a local variable **re-declared and re-initialized to 0 on every loop iteration** (not `static`), so it always starts fresh and becomes `1` each time.

Quick Tips for Zoho Round 1

- Expect **output-prediction** questions like these (not just theory) — practice tracing code by hand, not just recognizing patterns.
- Watch for **undefined behavior traps**: multiple modifications to a variable between sequence points (`x++ + ++x` style questions) are very common — know *why* they're UB even if you can guess GCC's typical output.
- Be solid on: pointer arithmetic, array-pointer duality, `strlen` vs `sizeof`, pass-by-value vs pass-by-address, `static` vs `auto` locals, macro pitfalls (missing parentheses, multiple evaluation), and struct copy-by-value semantics.
- Practice basic recursion (factorial, Fibonacci, palindrome, reverse array/string, prime check, GCD) — these often appear as both aptitude and coding-round questions.